2024 32nd International Conference on Electrical Engineering

Fast and Low Power Modified Carry Look-Ahead Adder

Sanaz Salem
dept. *Electrical and Electronic Engineering*
Shiraz University of Technology
Shiraz, Iran
S.Salem@sutech.ac.ir

Amir Hossein Owji
dept. *Electrical and Electronic Engineering*
Shiraz University of Technology
Shiraz, Iran
amir.owji003@gmail.com

***Abstract*— In this paper, we delved into examining the structure of the Brent-Kung Adder and proposed modifications to the gate architecture to design a 4-bit adder. The proposed model is simulated by using ISE Design Suite 14.7 software and compared its results in terms of propagation delay with the Kogge-Stone and Brent-Kung Adder. The designed model demonstrated superior performance in terms of propagation delay. From a transistor count perspective, the proposed model has approximately the same number of transistors as other models. Additionally, leveraging this proposed model and integrating it with a Carry Select Adder, an 8-bit adder is designed Considering the simulation results, it can be asserted that the designed model exhibits better performance in terms of propagation delay than other adders. Furthermore, in addition to these considerations investigated the power consumption is also investigated for all 4-bit and 8-bit adders. In the presented 8-bit model, the proposed design exhibits the lowest power consumption.**

Keywords— Brent-Kung Adder, Kogge-Stone Adder, Carry Look-Ahead, Delay, Parallel Prefix Adder.

I. Introduction

In numerous computing systems and various processor architectures, adders play a crucial role. They are not confined solely to the arithmetic logic unit; instead, they are integrated into various sections of the processor. Adders can be designed to accommodate various numerical representations, including binary coded decimal or excess code. Although adders can operate on different numerical systems, the most prevalent ones typically work with binary numbers. This holds even in scenarios where two complement or other complement representations are employed to denote negative numbers. Adders are employed not only for arithmetic calculations but also for their utility in computing addresses. [1,2,3].

The fast and accurate performance of a digital system relies on the efficiency of adders. Therefore, improving the performance of adders is a key factor in the design of digital systems, and contemporary designs strive to enhance the speed of adders. In this article, we have also proposed a model that exhibits the superior performance compared to other adders. Furthermore, an overview of common adders is provided, and the proposed model in the following section is discussed [1].

Adders can be broadly classified into two major categories, namely, parallel adders and serial adders [4].

II. Serial Adder

A. Ripple Carry Adder (RCA)

The Ripple Carry Adder (RCA) is a fundamental circuit used for adding two N-bit numbers. Comprising N full adders, it processes N-bit numbers sequentially. The carry input of each full adder, starting from the second one, is derived from the carry output of its preceding full adder. This sequential process, leading to a cascading effect, is why it's referred to as a Ripple Carry Adder. While the layout of the Ripple Carry Adder is straightforward, enabling rapid design, it is the slowest among adders due to each full adder waiting for the generation of the carry bit from its predecessor. The RCA can be cascaded in parallel for the addition of N-bit numbers, but this results in increased delay. Despite its low area consumption, the RCA exhibits high time delay. It remains a simple yet widely used approach, forming a cascade of full adders for N-bit addition. The Circuit is shown in Fig. 1 [5].

The sum and carry bits will be computed as follows:

$$S_i = A_i \; xor \; B_i \; xor \; C_i \qquad (1)$$

$$C_{i+1} = A_iB_i + B_iC_i + A_iC_i \qquad (2)$$

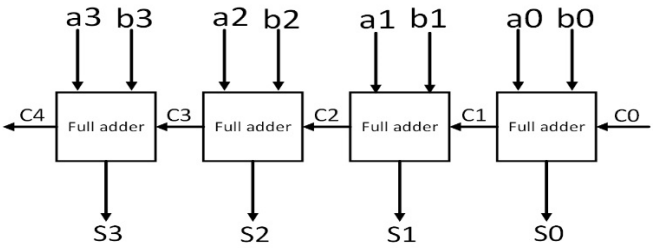


Fig. 1. Ripple Carry Adder[5]

B. Carry Save Adder (CSA)

Another efficient addition algorithm is the Carry Save Adder. Similar to a full adder, it generates two temporary outputs, namely Sum and Carry, from the two inputs. To obtain the sum bit, a bitwise XOR operation is performed, while the Carry bit is obtained through a bitwise AND operation on the two input numbers. The final step involves

2024 32nd International Conference on Electrical Engineering (ICEE) | 979-8-3503-7634-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICEE63041.2024.10668142

979-8-3503-7634-0/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:36:59 UTC from IEEE Xplore. Restrictions apply.

adding these temporary results by left-shifting the Carry bit by one position to align it with the Sum bit, ultimately producing the final answer[6].

C. Carry Select Adder (CSLA)

The conventional Carry Select Adder (CSLA) architecture strikes a balance between Ripple Carry Adder (RCA) and Carry Look-Ahead (CLA) methods, considering area and delay factors. In this structure, a pair of RCAs is utilized to generate sum and carry bits based on the anticipated carry-in (Cin) values, such as Cin = 1 and Cin = 0, Once the correct Cin is determined, a multiplexer selects the final sum and Cout bits. This dual pair of RCAs performs parallel addition, one with Cin = 1 and the other with Cin = 0. However, the conventional CSLA is not considered area-efficient due to the use of a dual pair of RCAs for sum and carry generation. To address this inefficiency, the conventional structure is optimized by replacing the lower RCA block with BEC (Binary-to-Excess-One Converter) to achieve reduced area and lower power consumption [3,7]. The Circuit is shown in Fig. 2 [4].

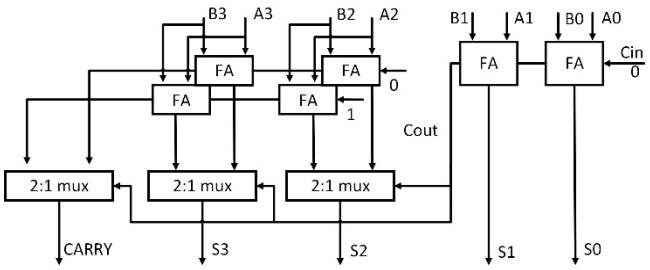


Fig. 2. Carry Select Adder [4]

D. Carry Look-Ahead Adder (CLA)

The fundamental concept behind carry look-ahead addition is an effort to simultaneously generate all incoming carries, eliminating the need to wait for the correct carry to propagate from the stage (FA) of the adder where it was initially generated [1]. The carry, C_i , propagation and generation signal produced at each stage is expressed as follows:

$$P_i = A_i \; xor \; B_i \qquad (3)$$

$$G_i = A_i . B_i \qquad (4)$$

$$C_i = G_i + P_i . C_{i-1} \qquad (5)$$

The equation can be understood as indicating the presence of a carry if one is either generated at that stage or propagated from the preceding stage. In simpler terms, a carry is generated when both operand bits are 1, and an incoming carry is propagated if one operand bit is 1, and the other is 0. Thus, using Gi and Pi to represent the generation and propagation at each stage, respectively, we have expressions for operand bits Ai and Bi and carry-in ci. These expressions enable the parallel calculation of all the carries from the operands.

As the number of bits in a group increases, the look-ahead carry logic becomes more intricate, leading to a higher proportion of time spent on the slower paths within each group, as opposed to the fast inter-group pathways facilitated by the look-ahead carry logic. Conversely, with fewer bits in a group, more groups must be traversed to span the entire range of a number, resulting in less acceleration [4].

In the RCA circuit, there is a delay in the signal propagation known as Propagation Delay (PD) due to the dependence of the sum and carry-out (Cout) bits of each adder stage on the preceding stage. A Carry Look-Ahead (CLA) adder offers a solution by mitigating the propagation delay, albeit at the expense of increased circuit area. Consequently, CLA adders are favored over RCA in applications where time sensitivity is critical. [7].

III. PARALLEL PREFIX ADDERS

The implementation of the Carry Look-Ahead Adder (CLA) architecture plays a pivotal role in the development of parallel prefix adders. By leveraging tree structure algorithms, these adders are designed to optimize the speed and overall efficiency of arithmetic operations. The incorporation of CLA architecture contributes to the accelerated performance of parallel prefix adders, making them indispensable components in high-performance arithmetic circuits. Their ability to significantly enhance operational speed positions them as key contributors to the efficiency and speed of complex arithmetic computations in various computational systems and applications [8].

Kogge Stone Adder (KSA), Brent Kung Adder (BKA), Han Carlson Adder (HCA), Hybrid Han Carlson Adder (HHCA) [10], Sklansky Adder [5], and Ladner-Fischer (LF) are different kinds of parallel prefix adder.

The structure of a parallel prefix adder comprises three stages:

- Pre-processing stage:

  In this stage, the generate and propagate signals for each pair of inputs A and B are computed.

- Carry generation Process:

  In this stage, carries for each bit are computed.

- Post-processing stage:

  This is the final step to compute the summation of input bits [9].

$$S_i = P_i \; xor \; C_{i-1} \qquad (6)$$

A. Kogge-Stone Adder

The Kogge-Stone adder is favored for wide adders due to its lower delay compared to other architectures. Each vertical stage of this adder produces Propagate and Generate bits, with Generate bits being generated in the final stage. These Generate bits are XORed with the initial Propagate bits after the input, producing the sum bits. Consequently, the Kogge-Stone adder operates using the parallel prefix method. It comprises two components: Black cells, responsible for computing both generate and propagate signals, and Grey cells, involved in generating signals essential for sum calculation in the post-processing stage. The Kogge-Stone adder excels in fast carry computation, making it suitable for high-speed applications. However, its advantages in speed come at the cost of increased area and power consumption [5,1,10]. Fig 3 illustrates the functions performed by the black and gray circle. and Fig 4 shows the structure of a 4-bit Kogge-Stone Adder [10].

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:36:59 UTC from IEEE Xplore. Restrictions apply.

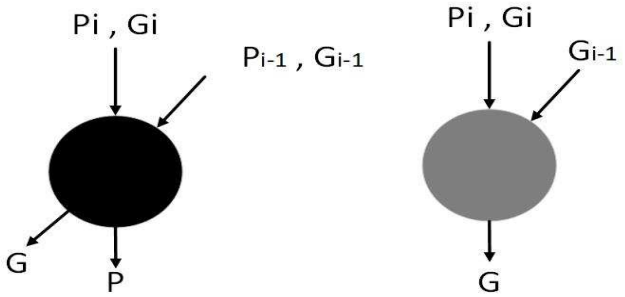


Fig. 3. Black and Gray cell [10]

$$G = G_i + P_i . G_{i-1} \qquad (7)$$

$$P = P_i . P_{i-1} \qquad (8)$$

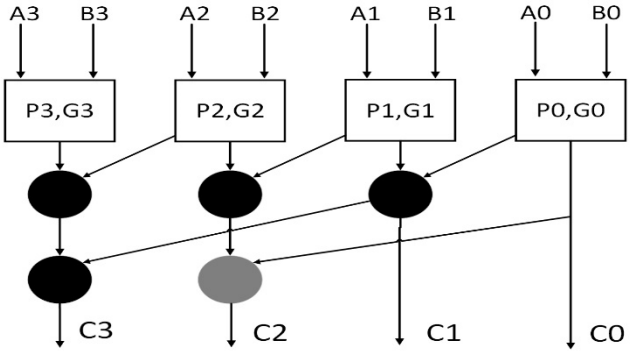


Fig. 4. Structure of a 4-bit Kogge-Stone Adder [10]

B. *Brent-Kung Adder*

The Brent-Kung adder stands as a widely acclaimed and extensively utilized addition to computational circuits. As a member of the parallel prefix adders, it belongs to the ultimate class of adders that leverage generate, and propagate signals. Notably, the Brent-Kung adder not only excels in terms of cost but also boasts reduced wiring complexity.

Structured for chip area reduction and simplified manufacturing, the Brent-Kung adder outpaces the traditional ripple carry adder in terms of speed. Its design emphasizes high regularity and minimal wiring complexity, leading to superior performance and a reduced chip area footprint. This adder significantly accelerates binary addition with its tree-like structure, demonstrating much lower wire complexity compared to the Kogge-Stone adder[9,11]. The structure of a 4-bit Brent Kung adder is given in Fig. 5.

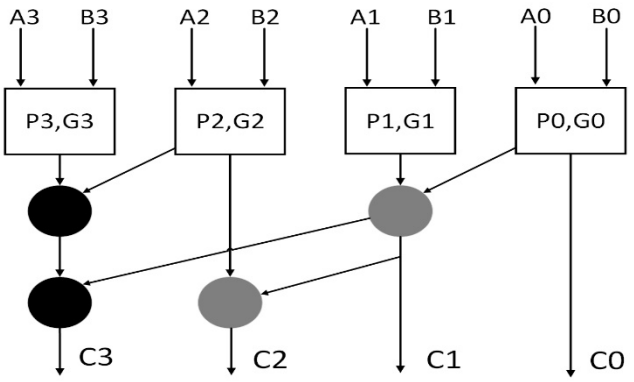


Fig. 5. Structure of a 4-bit Brent-Kung Adder [10]

C. *Comparison of the Brent-Kung and the Kogge-Stone Adder:*

Both Brent-Kung and Kogge-Stone adders are high-performance, parallel-prefix adders designed to markedly accelerate the addition of multiple binary numbers compared to traditional Ripple Carry Adders. Nevertheless, they vary in their approaches and present distinct advantages and disadvantages:

- Speed[12]:

  Kogge-Stone: Generally excels in speed for larger bit widths, leveraging its two-dimensional network that enables simultaneous carry propagation across all bits within each stage.

  Brent-Kung: Exhibits a slightly slower performance for larger bit widths as carries propagate through a tree-like network with multiple levels.

- Area and Complexity[12]:

  Kogge-Stone: Necessitates a greater number of logic gates and routing elements owing to its intricate network, resulting in a larger circuit area and heightened design complexity.

  Brent-Kung: Provides a smaller circuit area and a simpler design due to its tree-like network, rendering it more suitable for scenarios where area constraints are crucial.

- Power consumption[12]:

  Kogge-Stone might incur a slightly higher power consumption owing to its larger circuit size.

- Implementation difficulty:

  Implementing Brent-Kung is generally more straightforward due to its simpler design and logic equations.

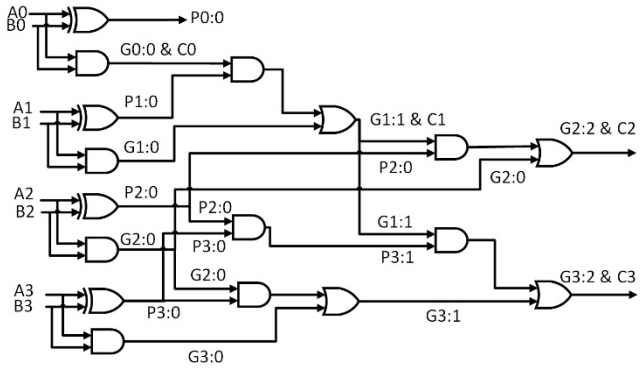


Fig. 6. 4-bit Brent-Kung Adder circuit [10]

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:36:59 UTC from IEEE Xplore. Restrictions apply.

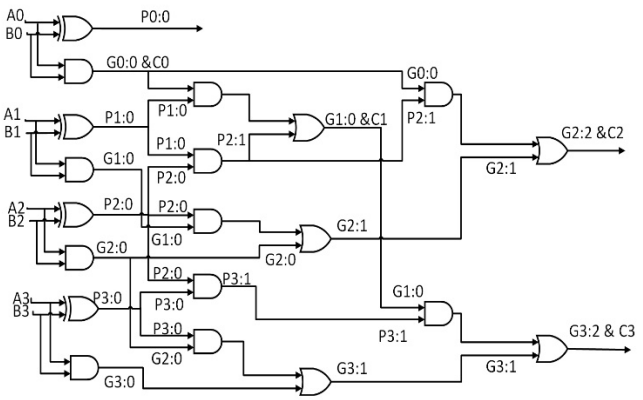


Fig. 7. 4-bit Kogge-Stone Adder circuit [10]

D. Proposed Modified Carry Look-Ahead Adder

The proposed modified 4-bit Carry Look-Ahead Adder is shown in Fig 8.

This structure is achieved by modifying the architecture of the Brent-Kung adder. By altering the third stage of this adder, it is managed to eliminate one of the stages. One of the changes involved relocating one of the OR gates from the third stage to a stage further ahead. By examining the inputs and outputs of another OR gate, which essentially had the value of C1 at its output, it is replaced with several AND gates to move this gate to a stage ahead and eliminate one of the stages. With the removal of one stage, a 3-input OR gate is used instead of the conventional 2-input OR gate in the parallel prefix adder.

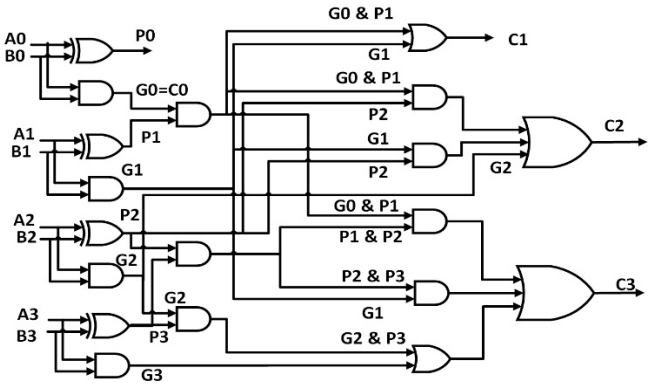


Fig. 8. Proposed 4-bit Modified CLA Adder circuit

As mentioned, the proposed architectural design distinguishes itself by incorporating one less stage compared to the conventional parallel prefix adders. In contrast to the Brent-Kung, where architectural modifications were implemented, it is chosen to utilize 104 transistors in 19 gates, deviating from the original 17 gates including 92 transistors. In the context of the Kogge-Stone adder, the gate count reaches 20 which includes 104 transistors. Consequently, it is expected a reduction in both power consumption and area when compared to the Kogge-Stone architecture.

Following that, the same proposed 4-bit design is extended to construct the structure of an 8-bit adder using a carry select adder. Essentially, a proposed model is integrated with a carry select adder to achieve a higher speed compared to the conventional carry select adder.

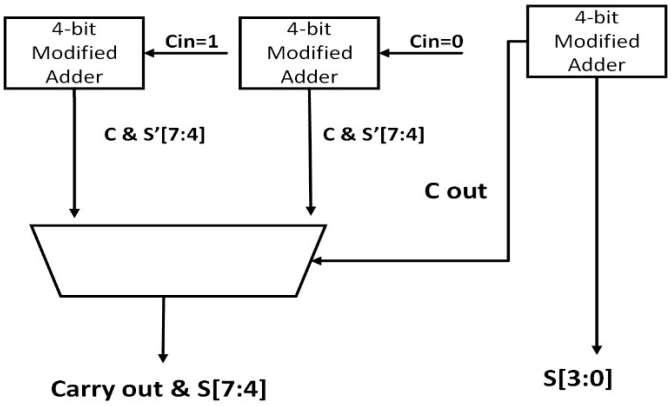


Fig. 9. Proposed 8-bit Modified CSA Adder circuit

In the conventional Carry Select adder, a Ripple Carry adder is employed to calculate. However, the use of a Ripple Carry adder introduces a notable limitation by increasing the delay. This is because the carry generated by one full adder must sequentially ripple through the next stage, contributing to a delay in the overall addition process. The diagram shown in Fig. 9 provides a clear depiction of the modified architecture.

IV. Simulation and Measurement Result

In the conducted simulation using ISE Design Suite 14.7 software and considering the propagation delay of various gate designs in the 4-bit proposed adder, the performance in terms of speed stood out with a propagation delay of 6.289ns. This outperformed other adders, such as the Kogge-Stone Adder with a delay of 6.413ns and the Brent-Kung Adder with a delay of 6.480ns. Additionally, the proposed 8-bit model with the Carry Select Adder, exhibited even higher speed, boasting a delay of 6.480ns, compared to a similar model simulated with the Brent-Kung Adder, which had a delay of 6.680ns. Furthermore, it surpassed the Kogge-Stone Adder, which had a delay of 6.602ns.

In terms of number of transistors, the Kogge-Stone Adder and the proposed adder has the highest number of transistors, while the Brent-Kung Adder has the fewest transistors.

The performance of these adders was also examined in the 180nm technology using Cadence software to evaluate parameters such as propagation delay and power consumption. In terms of speed, both proposed models demonstrated the best performance compared to other adders. Furthermore, among the 8-bit adders, the proposed design exhibited the lowest power consumption. In the case of the 4-bit adder, the Brent-Kung adder has slightly lower power consumption compared to the proposed 4-bit model. The power consumption of the Brent-Kung adder is 0.96 mW, while the proposed model has a power consumption of 0.97 mW.

Table I. Comparison Of Delay Measured for Spartan 3 and Number of Gates

Adder	4-bit Adder delay	8-bit Adder delay	Number of transistors
Kogge-Stone	6.413 ns	6.602 ns	104
Brent-Kung	6.480 ns	6.680 ns	92
Proposed Modified Adder	6.289 ns	6.480 ns	104

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:36:59 UTC from IEEE Xplore. Restrictions apply.

Table II. Comparison Of Delay and Power in 180nm technology

Adder/Parameters	Delay(ns)	Power(mW)
Kogge-Stone	0.556	1.09
Brent-Kung	0.587	0.96
Proposed Modified Adder	0.525	0.97
Kogge-Stone 8-bit	0.618	1.4
Brent-Kung 8-bit	0.647	1.36
Proposed Modified Adder 8-bit	0.589	1.22

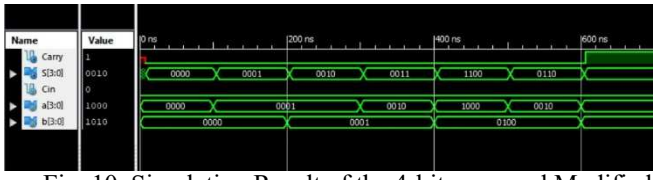


Fig. 10. Simulation Result of the 4-bit proposed Modified Adder

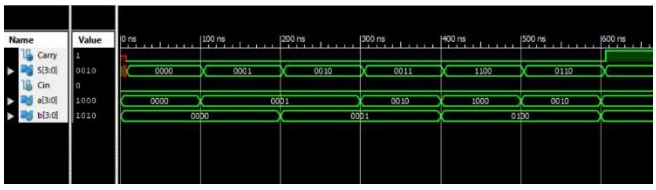


Fig. 11. Simulation Result of 4-bit Brent-Kung Adder

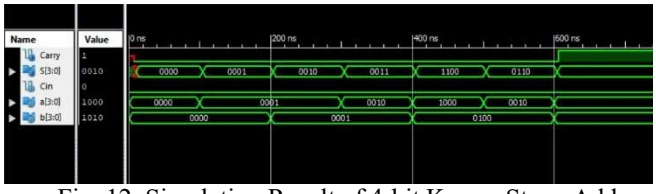


Fig. 12. Simulation Result of 4-bit Kogge-Stone Adder

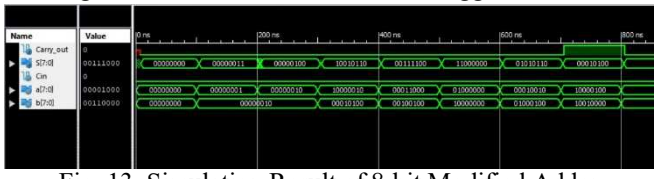


Fig. 13. Simulation Result of 8-bit Modified Adder

V. Conclusion

Serial adders like Carry Select and Carry Look-Ahead suffer from lower speed due to the need to wait for the previous stage's results. In contrast, parallel prefix adders exhibit higher speed as the processing occurs in parallel. Brent-Kung Adder and Kogge-Stone Adder are among these types of adders. In this article, a novel adder is proposed by examining the structure of the Brent-Kung Adder, and according to the simulation results, the proposed model demonstrated higher speed and lower area.

References

[1] S. Mythili, Senthil. K.M, D. Sathiesh Kumar, S. Soundarya, "High Speed Kogge Stone Adder Design," IJSRD - International Journal for Scientific Research & Development| Vol. 6, Issue 02, 2018.

[2] Pallavi Saxena, "Design of Low Power and High Speed Carry Select Adder Using Brent Kung Adder," International Conference on VLSI Systems, Architecture, Technology and Applications,2015.

[3] Pakkiraiah Chakali, Madhu Kumar Patnala, "Design of High Speed Ladner-Fischer Based Carry Select Adder", International Journal of Soft Computing and Engineering, Volume-3 Issue-1, March 2013.

[4] Raja Krishnamoorthy, S.Durgadevi, S.Maheshwari, "Area and delay carry select adder using Brent Kung architecture," IEEE International Conference on Electrical, Instrumentation and Communication Engineering,2017.

[5] Rahila K C, U. Sajesh Kumar, "A Comprehensive Comparative Analysis of Parallel Prefix Adders for ASIC Implementation," International conference on systems energy and environment,2019.

[6] Aritra Mitra, Amit Bakshi, Bhavesh Sharma, Nilesh Didwania, "Design of a High Speed Adder," International Journal of Scientific & Engineering Research, Volume 6, Issue 4, April 2015.

[7] Abhishek R Hebbar, Piyush Srivastava, Vinod Kumar Joshi, "Design of High Speed Carry Select Adder using Modified Parallel Prefix Adder," 8th International Conference on Advances in Computing and Communication (ICACC-2018).

[8] Pappu P. Potdukhe, Vishal D. Jaiswal, "Design of high speed carry select adder using brent kung adder," International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT) – 2016.

[9] Pappu P. Potdukhe1, Vishal D. Jaiswal, "REVIEW OF CARRY SELECT ADDER BY USING BRENT KUNG ADDER," International Journal of Advanced Research in Electronics and Communication Engineering (IJARECE) Volume 4, Issue 10, October 2015.

[10] Rakesh S , K. S. Vijula Grace, "A comprehensive review on the VLSI design performance of different Parallel Prefix Adders," Materials Today: Proceedings 11 (2019).

[11] Neema Zacharias , Prof. Lalu V , "Study of Approximate Multiplier with Different Adders," Proceedings of the International Conference on Smart Electronics and Communication (ICOSEC 2020).

[12] Vibhuti Dave, Erdal Oruklu, and Jafar Saniie, "Performance Evaluation of Flagged Prefix Adders for Constant Addition," IEEE International Conference on Electro/Information Technology,2006.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:36:59 UTC from IEEE Xplore. Restrictions apply.